

Computer Graphics – October 10th, 2024

Name: _____

Matriculation Number: _____

Please read the following instructions carefully:

- You have sixty (60) minutes to complete the exam.
- You can answer in German or in English.
- Check that the exam is complete (20 pages on 10 sheets) and readable!
- Please use a blue or black ballpen, not a red or green one. Do not use a pencil!
- Only text on the stapled sheets will be evaluated.
- If needed, we staple additional blank sheets.
- No other material except a calculator is allowed!

I have read and understood the instructions above.

Erlangen, October 10th, 2024 _____
Signature

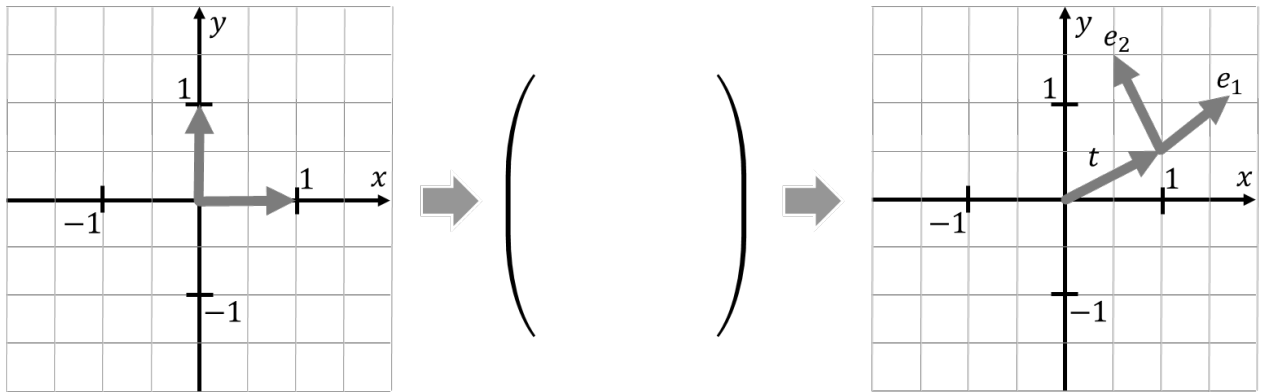
1	2	3	4	5	ALL
11	16	12	10	11	60

Grade:

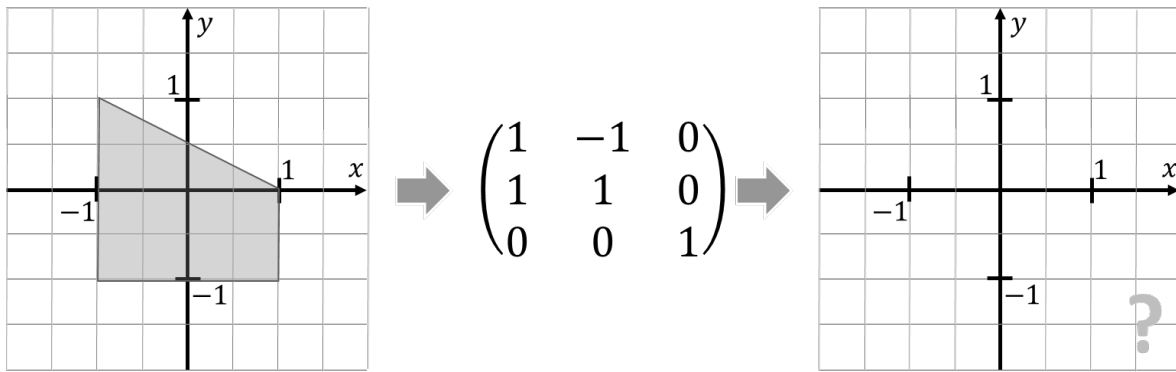
Assignment 1: Affine Transformations [11 points]

Affine mappings in **2D** can be expressed using **homogeneous coordinates** and 3×3 matrices.

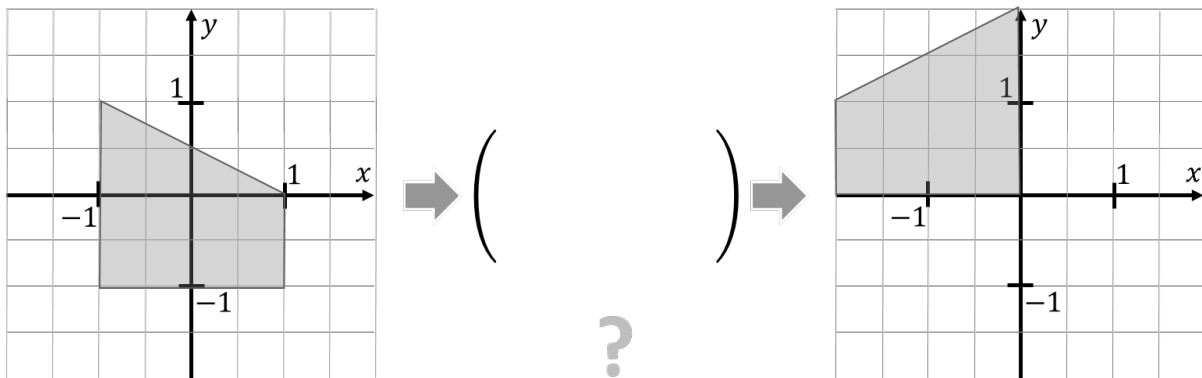
- a) **[2P]** Fill in the 3×3 matrix that maps the origin to $t = (t_x, t_y)$ and the x - and y -basis vector to $e_1 = (e_{1,x}, e_{1,y})$ and $e_2 = (e_{2,x}, e_{2,y})$.



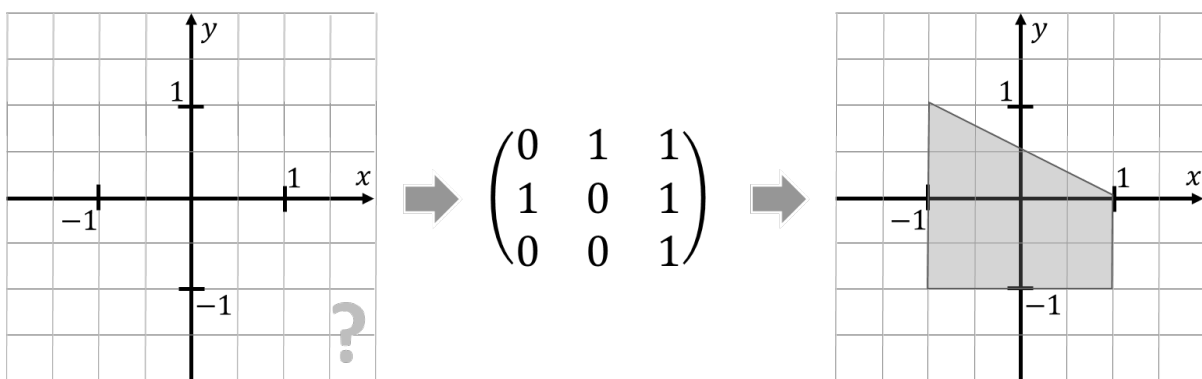
b) [2P] Sketch the result of the following mapping



c) [2P] Fill in the matrix that generates the shown mapping



d) [2P] Sketch the original shape on the left



e) **[3P] Concatenations of transformations.** In the lecture we defined several subclasses of 2D affine transformations. In this assignment, we look at the following classes:

- **affine** transformations
- **rigid** transformations
- **linear** transformations
- **rotations**
- **translations**

We now look at products of matrices of these classes. Given two matrices A and B , each of one of these subclasses. Write down the smallest subclass, to which the product $B \cdot A$ belongs.

A	B	$B \cdot A$
rigid	rigid	
translation	rotation	
affine	rigid	
rotation	rotation	
linear	translation	
rigid	translation	

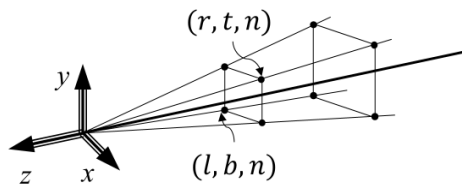
Assignment 2: Perspective [16 points]

a) [2P] Given is a pinhole camera with a field of view of 90° and an aspect ratio of 1:1. Write down the perspective matrix, using n and f for near and far. Below you can find the corresponding slides from the lecture.

$$P = \begin{pmatrix} & & & \\ & & & \\ & & & \\ & & & \end{pmatrix}$$

- **Perspective Projection:**

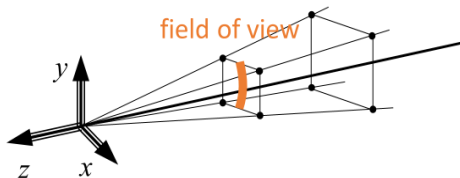
The depth range goes from n to f (near to far), and the image window (l, r, b, t) is defined on the near-plane



- $M_{perspective}(l, r, b, t, n, f) = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{l+r}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{b+t}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$

- Perspective Matrix usually defined by

- opening angle in y-direction: field of view in $y \rightarrow fovy$
- aspect ratio: ration of width over height $\rightarrow aspect$
- near and far plane $\rightarrow n, f$



- $-l = r = aspect \cdot n \cdot \tan \frac{fovy}{2}$
- $-b = t = n \cdot \tan \frac{fovy}{2}$

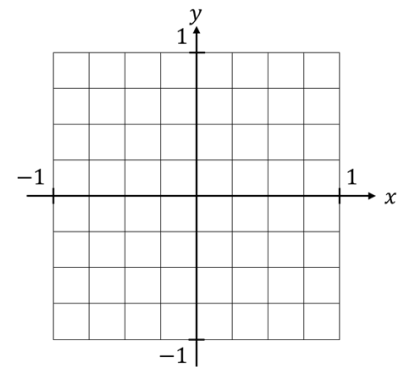
b) **[3P]** Write down the mapping of z in camera coordinates to depth after perspective. To which post-perspective depth values are $z_1 = -n$ and $z_2 = -f$ mapped?

c) **[4P]** Another camera has the following perspective matrix (which is different from a)

$$P = \begin{pmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & -3 & -4 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

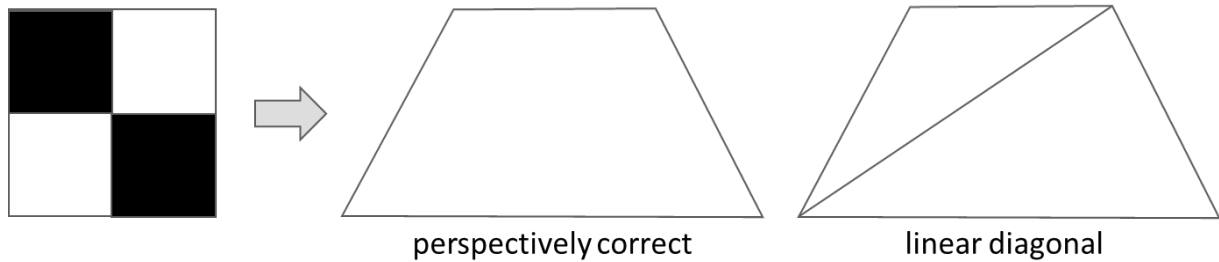
Map the rectangle with vertices $(1, -1, -1)$, $(1, -1, -2)$, $(1, 1, -2)$, $(1, 1, -1)$ using above matrix to image coordinates and sketch the 2D result.

$(1, -1, -1)$	→
$(1, -1, -2)$	→
$(1, 1, -2)$	→
$(1, 1, -1)$	→

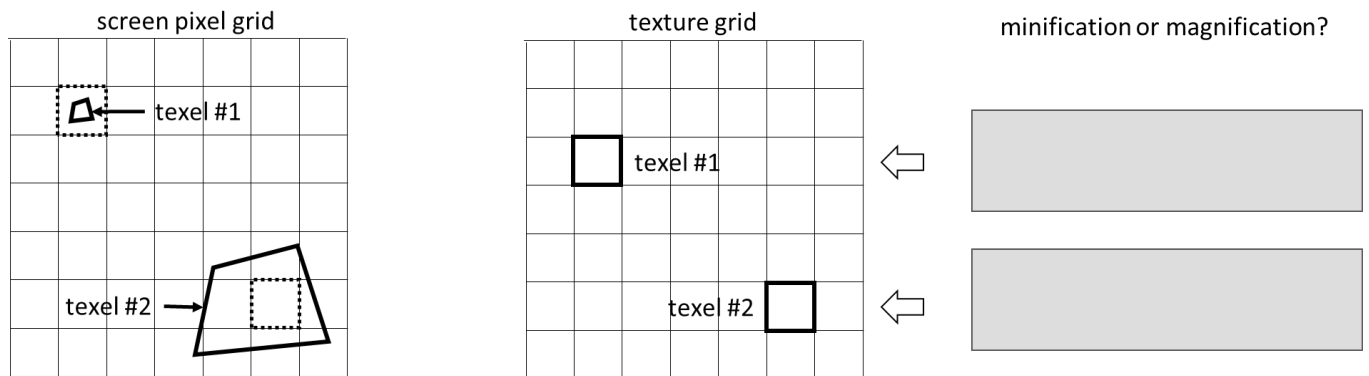


d) **[2P]** The above projection has a vanishing point at $(0,0)$. Which axis does this vanishing point belong to (x , y , or z)? Can you explain its position from the matrix P ?

e) [2P] Assume, we map a simple 2x2 checker board texture to a square, which is then mapped perspectively as shown below. Sketch how the texture is mapped to image space using perspectively correct interpolation (left), and with linear interpolation on two triangles, generated by subdivision of the square (right).



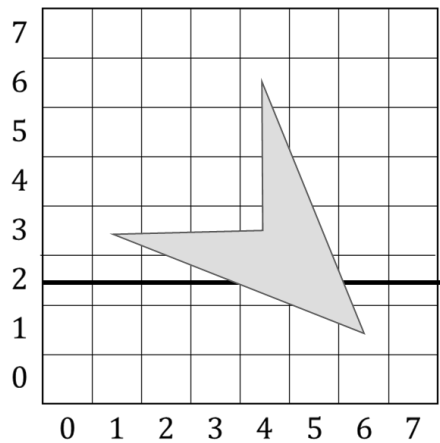
f) [3P] The following image shows two texels of a texture, as projected to screen space. Sketch the footprint of the marked pixels in the texture grid (shape is not so important, mostly its size and position). Note for both texels, whether they get minified or magnified.



Assignment 3: Rasterization [12 points]

In this assignment, we look at the rasterization of polygons using the scanline algorithm.

- a) [4P] Fill in the **edge table** for the following polygon with vertices $A = (1,3)$, $B = (6,1)$, $C = (4,6)$, $D = (4,3)$.



y_{lower}	x_{lower}	y_{upper}	$1/m$	$next$

- b) [2P] Fill in the **active edge table** below for the scanline $y_{scan} = 2$

x_{inter}	y_{upper}	$1/m$	$next$

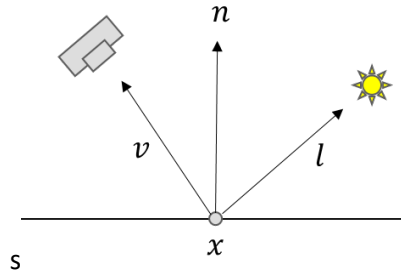
- c) [1P] How many edges are in the active edge table for the scanline $y_{scan} = 3$?

d) **[2P]** Sketch an algorithm in pseudocode to find out, whether a 2D-point x is inside a polygon $p_1 \dots p_n$ or not. You may use the function `intersectRay( $\mathbf{x}, \mathbf{p}, \mathbf{q}$ )` that returns “1”, if a ray from $\mathbf{x}$ in fixed direction (1,0) intersects edge $(\mathbf{p}, \mathbf{q})$ and “0” otherwise.

d) **[3P]** Sketch an algorithm in pseudocode to rasterize a circle with center (m_x, m_y) and radius r . To this end, compute its bounding box, iterate over all pixels inside, and set a pixel if it lies within the circle.

Assignment 4: Phong Lighting & Shading [10 points]

Given is a surface point x with normal n , illuminated by a light source in direction l , and looked at from direction v , as shown in the image below. We now want to compute the lighting at x under incident light I_{in} , a diffuse coefficient of k_{diff} , a specular coefficient of k_{spec} , and a shiny exponent of n_{shiny} .



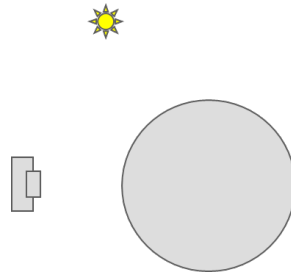
- a) **[1P]** Write down the formula to compute the diffuse lighting, using above defined variables. Use dot products to compute the cosine; you can assume that all direction vectors are normalized.

$$L_{diff} =$$

- b) **[2P]** Sketch the reflection vector r and write down the formula to compute the specular lighting:

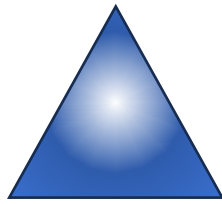
$$L_{spec} =$$

c) **[2P]** In the following setup, mark the points on the sphere, where diffuse and specular lighting becomes maximal:



d) **[1P]** Given that the light source has an intensity of I , how is the incident light I_{in} computed with physically correct attenuation? Define the used variables.

e) [2P] The following glossy triangle is lit using a point light. Which shading technique has been used to render the triangle: Flat, Gouraud, or Phong shading? Explain your answer!



f) [2P] The following code snippets show a vertex and fragment shader, implementing diffuse lighting. Does this shader pair realize Gouraud Shading or Phong shading? Explain your answer!

```
1 uniform mat4 M,V,P;  
2 uniform mat3 N;  
3 attribute vec3 pos,nrm;  
4 varying vec3 n;  
5  
6 void main() {  
7     gl_Position = P*V*M*vec4(pos,1.0);  
8     n = normalize(N * nrm);  
9 }
```

```
1 precision highp float;  
2  
3 uniform vec3 diff,light;  
4 varying vec3 n;  
5  
6 void main() {  
7     gl_FragColor = vec4(diff*dot(n,light),1);  
8 }
```


Assignment 5: Run-time Complexity [11 points]

In this assignment we compare run-time complexities of different rendering algorithms.

We assume we want to rasterize a scene with T triangles and V vertices into an image with P pixels. All triangles have roughly the same size of S pixels. We assume that we are in a closed scene, i.e. there are no empty background pixels and every pixel is covered by the scene.

a) [4P] Rasterization:

In the following table, we want to summarize the number of required calls of certain steps of the rendering pipeline. “Lighting Computations” is the number of evaluations of the lighting model (e.g., Phong), assuming a single light source. Note that deferred shading requires two render passes, **in this assignment we only consider the more costly calls from the second lighting pass!**

Fill in the missing fields in the following table:

Algorithm	Vertex Shader Calls	Pixel Shader Calls	Lighting Computations	Depth tests
Rasterization with Gouraud Shading	V			
Rasterization with Pixel Shading		$T \cdot S$		
Lighting Pass of Deferred Shading (second pass only)	4 (screen-space-aligned quad)			
Rasterization with 4x SSAA			$4 \cdot T \cdot S$	

Next, we look into the run-time complexity of ray tracing algorithms and the resulting ray trees in different configurations. Below you can find the corresponding lecture slide:

Basic Ray Tracing

```
for each pixel p
  compute eye ray
  c = raytrace(ray,0)
  set pixel p to color c

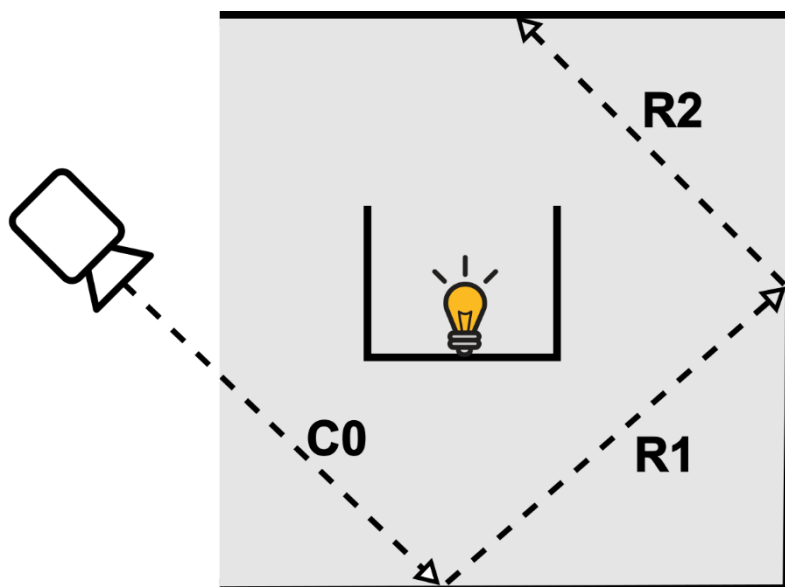
raytrace(ray,depth)
  hit = intersect(scene,ray)
  c = black;
  if (hit.shader.isReflective() and depth < maxdepth)
    reflray = compute reflection ray
    c += raytrace(reflray,depth+1) * reflcolor
  if (hit.shader.isRefractive() and depth < maxdepth)
    refray = compute refraction ray
    c += raytrace(refray,depth+1) * refrcolor
  shadowRay = compute shadow ray
  if (intersect(scene,shadowRay)
    c += hit.shader.ambientColor
  else
    c += hit.shader.computePhongColor();
  return c;
```

Computer Graphics 2023/24 - Ray Tracing Secondary Effects

6

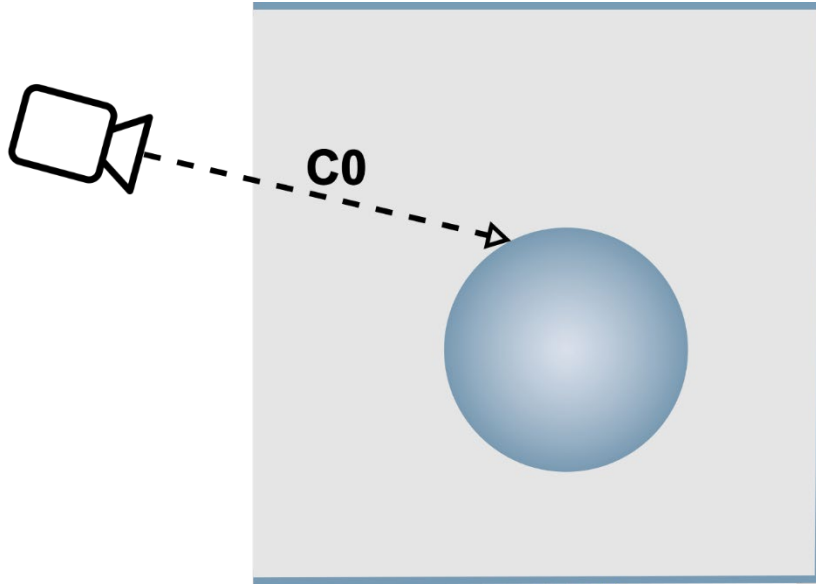
b) [2P] Ray Tracing - Shadow Rays

In the following figure, we already traced one camera ray **C0** and two reflection rays **R1** and **R2** ($\text{maxdepth} == 2$, no refraction). To compute the illumination at each hit point, we need to trace additional shadow rays. Sketch all **shadow rays** to the given **point light source** in this ray tree and mark their **intersections** with the scene, if applicable.



c) [2P] Ray Tracing - Reflection and Refraction Rays

We now render a scene completely made of glass, i.e., we must consider both reflection and refraction at each intersection with the scene! Sketch the resulting ray tree generated by a call to `raytrace(C0, 0)` with `maxdepth == 2`. Please leave out the shadow rays!



d) [3P] Ray Tracing - Intersection Tests

We now additionally consider shadow rays while rendering our scene made of glass, i.e., we trace a **reflection**, **refraction**, and **shadow** ray for each hit point.

How many **intersection tests** with the scene are performed by one call of `raytrace(ray, 0)` for `maxdepth == 3` ? Hint: For `maxdepth == 1`, the answer would be "6".

#Ray-scene intersections =

How many **intersection tests** with the scene are performed for a general ray tree of depth N (`maxdepth == N`) ?

#Ray-scene intersections =

How many **intersection tests** with the scene are performed by one call of `raytrace(ray, 0)` for `maxdepth == 3` if we have two light sources (code above needs to be extended) ?

#Ray-scene intersections =

